

PREPARANDO O TERRENO • FERRAMENTAS E PYTHON

Ambiente de Desenvolvimento em Python

Antes de treinar modelos, todos precisam do mesmo ponto de partida: um ambiente configurado, as ferramentas certas e o Python essencial na ponta dos dedos.



Ambiente pronto é **meio caminho andado** — hoje deixamos tudo funcionando antes de codar de verdade.

Três frentes para começar a praticar

Já entendemos **o que** é aprendizado de máquina. Antes dos algoritmos, garantimos que todos consigam **rodar código**.

Configurar o ambiente

Python, pip e ambientes virtuais funcionando na sua máquina.

Conhecer as ferramentas

Jupyter Lab, Google Colab e VS Code — onde vamos escrever código.

Revisar o Python

a base da linguagem que sustenta todos os próximos conteúdos.

Meta da aula: ao final, todos com o ambiente rodando e confortáveis para escrever os primeiros scripts.

O NÚCLEO DO AMBIENTE

Quatro peças essenciais

Tudo começa com a linguagem e o mínimo para instalar e isolar bibliotecas.

Python ≥ 3.6

a linguagem fundamental que usaremos em todo o curso.

pip

o gerenciador que instala e atualiza bibliotecas nos seus projetos.

virtualenv

cria ambientes virtuais, isolando dependências e evitando conflitos.

IPython

um terminal interativo mais rico que o console padrão do Python.

Com essas quatro peças você já **instala, isola e executa** qualquer projeto Python.

ONDE ESCREVER E O QUE IMPORTAR

Ambientes de trabalho e bibliotecas

• Ambientes e IDEs

Jupyter Lab

ambiente web interativo para notebooks — ideal para análise de dados.

VS Code

editor leve, poderoso e altamente personalizável.

Google Colab

Jupyter na nuvem, gratuito e com acesso a GPU/TPU.

PyCharm

IDE completa, voltada ao desenvolvimento profissional.

• Bibliotecas essenciais

NumPy

computação numérica com arrays e matrizes de alta performance.

Matplotlib • Seaborn

visualizações estáticas e gráficos estatísticos.

Pandas

manipulação e análise de dados com DataFrames.

Scikit-learn • Plotly

aprendizado de máquina e gráficos interativos.

PARTE 1 • MÃOS NO TERMINAL

Preparando o ambiente

Do download do Python ao ambiente virtual com todas as bibliotecas instaladas.

Instalando o essencial

- 01 **Python** — baixe e instale a versão mais recente pelo site oficial (python.org). Marque a opção **Add to PATH** no Windows.
- 02 **pip** — já vem com as versões recentes do Python. Se precisar, instale separadamente pelo guia oficial.
- 03 **virtualenv** — para isolar projetos e evitar conflitos de dependências: `pip install virtualenv`

Dica: instalar o Python é o único passo que depende do sistema operacional. A partir daqui, os comandos são iguais no terminal de qualquer máquina.

CONFIRMANDO QUE FUNCIONA

Verificando a instalação

```
# abre o console interativo do Python
$ python
# para sair:
>>> quit()      # ou CTRL+D

# confere a versão do pip
$ pip --version
pip 24.0 from ... (python 3.12)
```



Se o console abre e mostra a versão, o **Python está no PATH** e pronto.



pip = "Pip Installs Packages": instala e gerencia bibliotecas do repositório oficial **PyPI**.

ISOLANDO O PROJETO

Criando um ambiente virtual

```
# 1 · criar (gera a pasta myenv)
$ virtualenv myenv

# 2 · ativar
$ .\myenv\Scripts\activate.bat # Windows
$ source myenv/bin/activate   # Linux / macOS

# 3 · desativar
(myenv) $ deactivate
```

01 Criar gera uma pasta com uma cópia isolada do Python.

02 Ativar faz o terminal usar esse ambiente — repare no `(myenv)` no prompt.

03 Desativar volta ao Python global do sistema.



Com o ambiente **ativado**, os pacotes ficam só nele — sem afetar a instalação global.

PIP NO DIA A DIA

Gerenciando pacotes Python

```
# com o ambiente virtual ativado
(myenv) $ pip list           # listar
(myenv) $ pip install nome_do_pacote # instalar
(myenv) $ pip uninstall nome_do_pacote # remover
```

Os pacotes vão para `myenv/lib/python3.x/site-packages/` — dentro do ambiente, não no sistema.

Pacotes essenciais do curso

<code>pip install ipython</code>	<code>pip install numpy</code>
<code>pip install pandas</code>	<code>pip install sklearn</code>
<code>pip install jupyterlab</code>	<code>pip install matplotlib</code>
<code>pip install seaborn</code>	<code>pip install plotly</code>

NOSSO AMBIENTE PRINCIPAL

Jupyter Lab

Ambiente web interativo para desenvolver, executar e compartilhar código. Ideal para **notebooks** que combinam código, visualizações e texto.

Código e texto juntos

alterne entre código Python e texto em Markdown, criando documentos dinâmicos e explicativos.

Feito para dados

perfeito para análise, experimentos, prototipagem rápida e documentação de projetos.

É onde vamos **rodar quase tudo** ao longo da disciplina — do primeiro gráfico ao modelo treinado.

PRIMEIROS PASSOS

Iniciando e usando o notebook

• Como iniciar

- 01 No terminal, digite `jupyter lab` — a interface abre no navegador.
- 02 Navegue até a pasta do projeto e crie um novo notebook `.ipynb` para começar.

• Tipos de célula

Código

escreve e executa comandos Python, mostrando o resultado na hora.

Markdown

anotações, descrições, fórmulas e explicações formatadas.

Um notebook é um **documento executável**: o raciocínio, o código e o resultado ficam lado a lado — perfeito para estudar e apresentar.

Ambientes alternativos

Google Colab

Jupyter na nuvem do Google, com GPU/TPU grátis. Ambiente pronto, notebooks no Drive e compartilhamento fácil.

VS Code

editor leve e personalizável, com ótimas extensões para Python. Configure-o para usar o interpretador do seu ambiente virtual.

PyCharm

IDE completa e robusta da JetBrains, com recursos avançados para desenvolvimento profissional.

Sem instalar nada? O Google Colab roda no navegador com as principais bibliotecas já prontas — ótimo para quem tem dificuldade no setup local.

PARTE 2 • A LINGUAGEM

Fundamentos de Python

Uma revisão rápida do essencial — a base para todo o código do curso.

Variáveis, tipos e condicionais

- **Case sensitive:** nome $\neq$ Nome.
- **Fortemente e dinamicamente tipada** — o tipo é inferido, mas não muda sozinho.
- **Conversão:** `int()` · `float()` · `str()`

```
# atribuição simples e múltipla
x = 10
x, y = 10, 20

# condicionais — a indentação define o bloco
if numero % 2 == 0:
    tipo = "par"
else:
    tipo = "ímpar"

# condicional em uma linha
tipo = "par" if numero % 2 == 0 else "ímpar"
```

Em Python, a **indentação não é estética** — é ela que define onde cada bloco começa e termina.

ORGANIZANDO O CÓDIGO

Funções e ponto de entrada

```
def cumprimentar(nome):  
    """Retorna uma saudação personalizada."""  
    return f"Olá, {nome}!"  
  
def main():  
    """Função principal do programa."""  
    msg = cumprimentar("Turma de ML")  
    print(msg)  
  
if __name__ == "__main__":  
    main() # executa só quando rodado direto
```



def define funções — blocos reutilizáveis. A indentação marca o escopo.



main() por convenção é o ponto de entrada que organiza a lógica central.



`__name__ == "__main__"` garante que **main()** só rode ao executar o arquivo direto, não ao importá-lo.

Loops e listas

• Loops

```
# percorre uma sequência
for i in range(5):
    print(i)      # 0,1,2,3,4

# repete enquanto a condição for verdadeira
contador = 0
while contador < 5:
    contador += 1
```

• Listas

```
numeros = [1, 2, 3, 4]

numeros.append(5)  # adiciona ao final
numeros.pop()      # remove o último
numeros.sort()     # ordena

# list comprehension
quadrados = [x**2 for x in range(10)]
```

Strings e slices

• Strings

```
texto = "Python é incrível"

texto.upper()          # maiúsculas
texto.split()          # lista de palavras
texto.replace("incrível", "top")

texto[:6]              # "Python"
```

• Slices · listas e strings

```
lista = [0, 1, 2, 3, 4, 5]

lista[1:4]             # [1, 2, 3]
lista[::2]             # [0, 2, 4]
lista[::-1]            # invertida
```

A sintaxe `[início:fim:passo]` funciona igual em listas e strings.

Tuplas e dicionários

• Tuplas · imutáveis

```
numeros = (1, 2, 3)

# numeros[0] = 5 → erro! imutável

primeiro = numeros[0]
lista = list(numeros) # converte p/ lista
```

• Dicionários · chave → valor

```
usuario = {
    "nome": "Ana",
    "idade": 25,
}

usuario["nome"] # "Ana"
usuario["email"] = "ana@..." # adiciona

for chave, valor in usuario.items():
    print(f"{chave}: {valor}")
```

Módulos, arquivos e classes

• Módulos

```
import math
from random import randint

math.pi
randint(1, 6)
```

• Arquivos

```
with open('dados.txt') as f:
    conteudo = f.read()

with open('saida.txt', 'w') as f:
    f.write('Olá!')
```

• Classes

```
class Pessoa:
    def __init__(self, nome):
        self.nome = nome

    def ola(self):
        return self.nome

p = Pessoa("Maria")
```

Esses tópicos são a base para aplicações mais complexas — **análise de dados, sistemas e inteligência artificial.**

PARTE 3 • MÃO NA MASSA

Vamos praticar

Ambiente configurado e Python revisado —
hora de escrever os primeiros scripts.

Prepare o ambiente e teste o Python

PONTO DE PARTIDA

Um terminal aberto e o Python instalado na máquina.

- 01 Crie e **ative** um ambiente virtual chamado `env-ml`.
- 02 Instale **numpy**, **pandas** e **jupyterlab** nele.
- 03 Abra o **Jupyter Lab** e crie um notebook novo.

• No notebook

- a Escreva uma função **main()** que imprima seu nome.
- b Crie uma **lista** de 5 números e retorne os **quadrados** com list comprehension.
- c Monte um **dicionário** com seus dados e itere sobre ele com `.items()`.

Entrega: o notebook **.ipynb** com as três células rodando.

SÍNTESE

O que levamos desta aula

- 01 O **ambiente virtual** isola as dependências de cada projeto — crie, ative e instale dentro dele.
- 02 O **pip** instala bibliotecas do PyPI; o **Jupyter Lab** é onde vamos escrever e rodar código.
- 03 Revisamos o **Python essencial**: variáveis, condicionais, funções, loops e estruturas de dados.
- 04 Com tudo funcionando, estamos prontos para **manipular dados de verdade**.